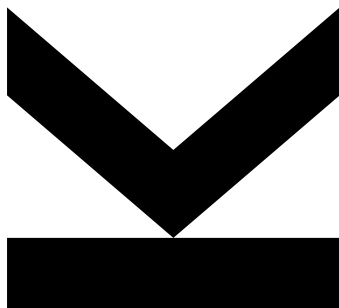


**Simon Grundner**  
Institute for Signal  
Processing

@ k12136610@students.jku.at  
🌐 <https://jku.at/isp>

October 3, 2025

# 03 - Pulse Width Modulation



**Hardwaredesign PR - Exercise 03**

**Semester 2024W**

## **Contents**

|                            |           |
|----------------------------|-----------|
| <b>1. Delta ADC</b>        | <b>3</b>  |
| <b>2. Delta ADC</b>        | <b>4</b>  |
| <b>3. PWM</b>              | <b>8</b>  |
| <b>4. Strobe Generator</b> | <b>10</b> |
| <b>5. Counter</b>          | <b>12</b> |

## 1. Delta ADC

### Delta ADC (Nachlaufumsetzer)

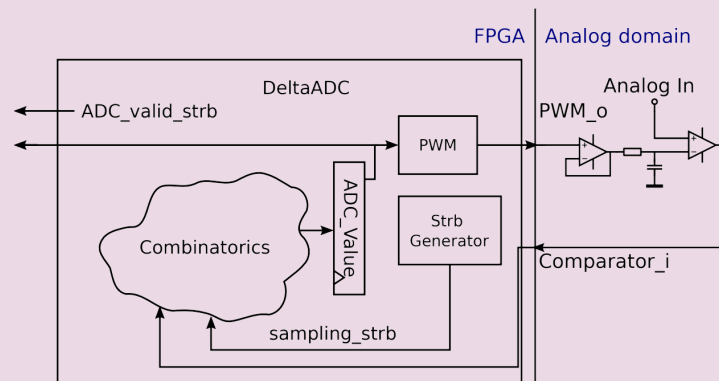


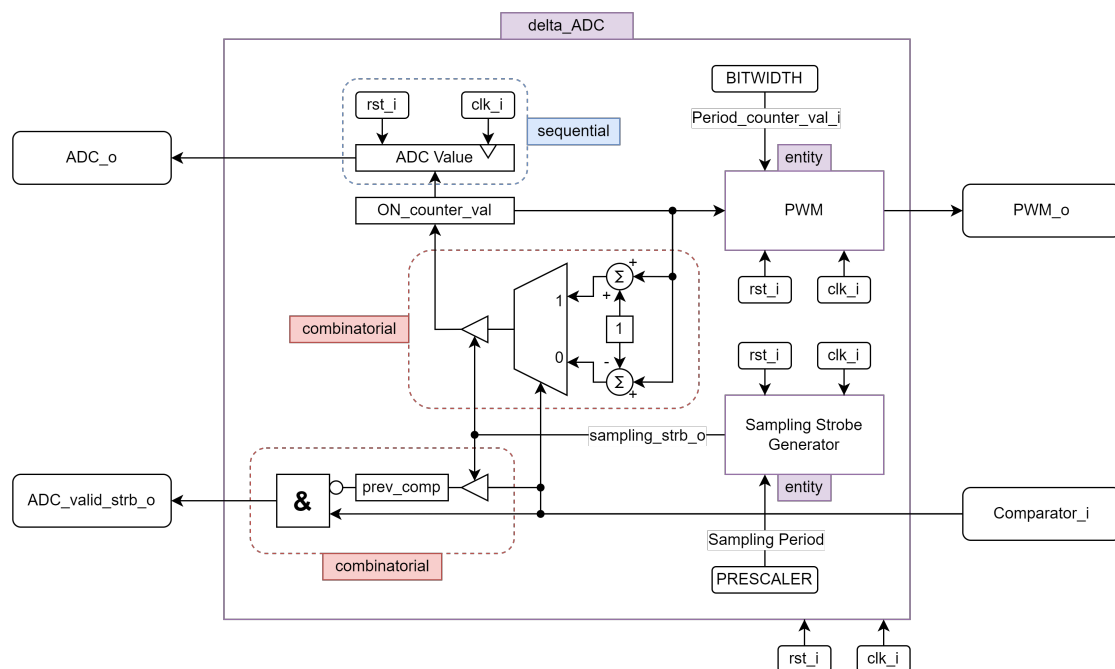
Figure 1: Delta ADC Block

In this exercise, we will start the digital design of a delta ADC, schematically shown in Figure 1. It works like the following. A PWM Signal generated by the FPGA is output through an analog low pass filter (more details on the analog part will follow in a future exercise), resulting in a voltage level that is compared to the analog input via an analog comparator. This comparator outputs a zero if the analog input is lower than the comparison voltage level and a one otherwise. In the digital domain the comparator signal (Comparator i) is used to increase the ADC value in case Comparator i is one and decrease the ADC value in case Comparator i is zero. A change of the ADC value should only occur at rising clock edges where the sampling strobe (sampling strb) generated by a strobe generator is one. This will lead to a sampling that is equidistantly spaced in time with the sampling frequency defined by the frequency of sampling strb. The ADC value is feed to the PWM to define the ON counter val i. This way a simple control loop is closed, allowing the ADC value to adapt to analog signal. This way the ADC signal will follow the analog signal with its value proportional (up to an adjustment error) to the analog input.

## To-Do-List

- Draw a block diagram of the required combinatorics. Do you need to have a finite state machine? If you do, please write a state diagram.
- Draw the required blocks to the block diagram for implementing the signal ADC valid strb.
- Write the entities and architectures that are required in VHDL. As you did not design the strobe generator yet it is suggested that you start with the strobe generator. Find meaningful generics such that the frequency of the strobe generator is adjustable (you might set the time between strobes to a few clock cycles for the simulation, however for future use several thousands of clock cycles might be required). Use one .vhd file for each entity and one .vhd file for each architecture.
- Write a testbench in VHDL (extra file). Simulate the system using Modelsim and hand in screenshots showing the output of the waveform window. Find meaningful test cases. As you are not able to simulate the analog domain, your testbench can only simulate the behavior of the comparator input. What are meaningful testcases for Comparator i to adequately simulate the designs behavior.
- Discuss the results of the simulation in a few sentences and answer the questions.

## 2. Delta ADC



Entity\_\_\_\_\_

## Architecture

Die ADC-Architektur verwendet die im Weiteren beschriebenen Entities PWM und Sampling-Strobe-Generator. Ansonsten wird er ADC durch drei weitere Prozesse beschrieben.

## Signale

---

```

1 constant MAX_COUNTER_VAL : unsigned(BITWIDTH-1 downto 0) := (others => '1');
2 signal ON_counter_val : unsigned(BITWIDTH-1 downto 0) := (others => '0');
3 signal sampling_strobe : std_ulogic := '0';
4 signal prev_comp : std_logic := '0';

```

---

(1) reg\_seq : **process** (clk\_i, rst\_i)

Der sequenzielle Prozess bestimmt das Clock- und Reset-Verhalten. Bei der steigenden Flanke wird dem ADC-Ausgang der nächste, kombinatorisch bestimmte ADC-Zustand zugewiesen, welcher gleichzeitig der Duty-Cycle Counter-Wert ON\_counter\_val des PWM-Generators ist. Beim Reset wird der ADC-Ausgang auf '0' gesetzt.

(2) adc\_comb : **process** (sampling\_strobe, Comparator\_i)

Hier wird die kombinatorik beschrieben, welche zur Bestimmung des ADC-Wertes notwendig ist. Unter Voraussetzung, dass die sampling\_strobe auf '1' ist, gilt folgendes: Falls der vom Komparator gelieferte Eingangswert Comparator\_i auf '1' ist, wird der Arbeitszyklus um 1 inkrementiert, sonst dekrementiert. Außerdem wird mit jeder sampling\_strobe der momentane Komparator-Zustand gespeichert, welches für die Validierung des ADC-Wertes Relevant ist.

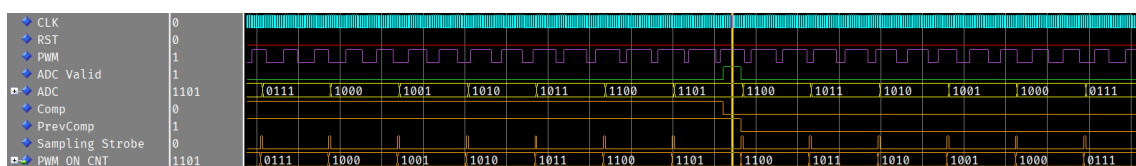
(3) validate\_comb : **process** (Comparator\_i, prev\_comp)

Der ADC verfügt über einen ADC\_valid\_strb\_o Ausgang, welcher signalisiert ob der tatsächliche ADC Wert am Ausgang anliegt, also der Einschwingvorgang beendet ist. In dieser Implementierung liegt dieser Zustand, vor falls sich der Komparator-Wert umkehrt. Ist also der Komparator-Wert schon seit einigen Sampling-Perioden auf '1' und wird plötzlich '0' (und umgekehrt), so hat der PWM-Mittelwert in der letzten Sampling-Periode den Analogen Wert erreicht und am ADC-Ausgang liegt näherungsweise der richtige Wert vor.

## Simulation

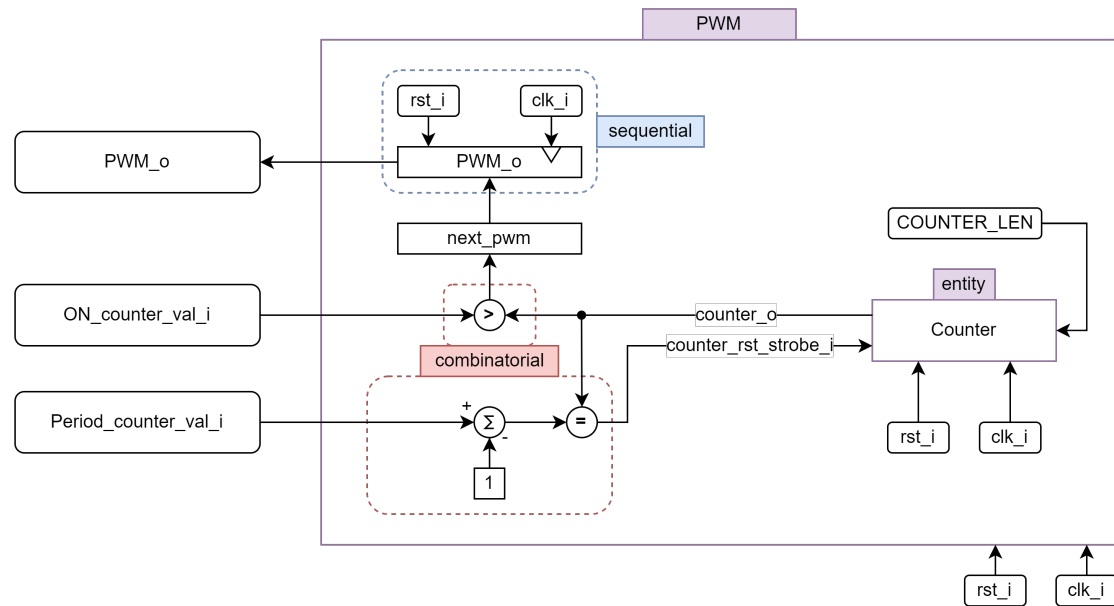
## ADC-Test

Hier wird der Komparator-Input Emuliert, indem er im Stimulus Prozess nach einer gewissen Zeit auf 1 gesetzt wird. Während dieser Zeit ist zusehen, wie der Duty-Cycle des PWM Signals zunimmt und wie erwartet mit dem Umschalten des Komparator Eingangs wieder Abnimmt. In der Sampling Periode kurz nach dem Umschalten ist auch zu erkennen wie die ADC-Valid-Strobe high wird.



Hier wäre der ADC-Wert 1101.

### 3. PWM



Entity \_\_\_\_\_

Architecture \_\_\_\_\_

Die Architektur des PWM-Generators besteht aus einem counter-Entity und drei Prozessen.

Signale:

```

1  signal count          : unsigned(COUNTER_LEN-1 downto 0);
2  signal count_reset_flag : std_ulogic := '0';
3  signal next_pwm        : std_ulogic;

```

(1) reg\_seq : **process** (clk\_i, rst\_i)

Der sequenzielle Prozess bestimmt das Clock- und Reset-Verhalten. Bei der steigenden Flanke wird dem PWM-Ausgang der kombinatorisch bestimmte nächste PWM-Zustand next\_pwm zugewiesen. Beim Reset wird der PWM-Ausgang auf '0' gesetzt.

(2) pwm\_comb : **process** (count, ON\_counter\_val\_i)

Der kombinatorische Prozess setzt den nächsten PWM-Ausgangswert auf '1', falls der Counter-Wert des Counter-Entities die Duty-Cycle-Schwelle ON\_counter\_val\_i noch nicht erreicht hat. Wird dieser Wert überschritten, wird next\_pwm auf '0' gesetzt.

(3) clr\_comb : **process** (count, Period\_counter\_val\_i)

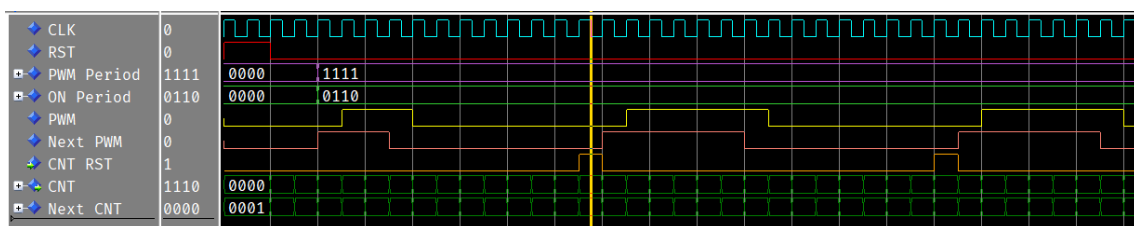
Dieser kombinatorische Prozess ist für das Zurücksetzen der Counters nach einer Periode zuständig. Dafür wird eine count\_reset\_flag, welche mit der Reset-Strobe des Counters verbunden ist, auf '1' gesetzt. Somit wird im Counter-Entity ein synchroner Reset verursacht. Ansonsten ist die count\_reset\_flag auf '0'.

## Simulation

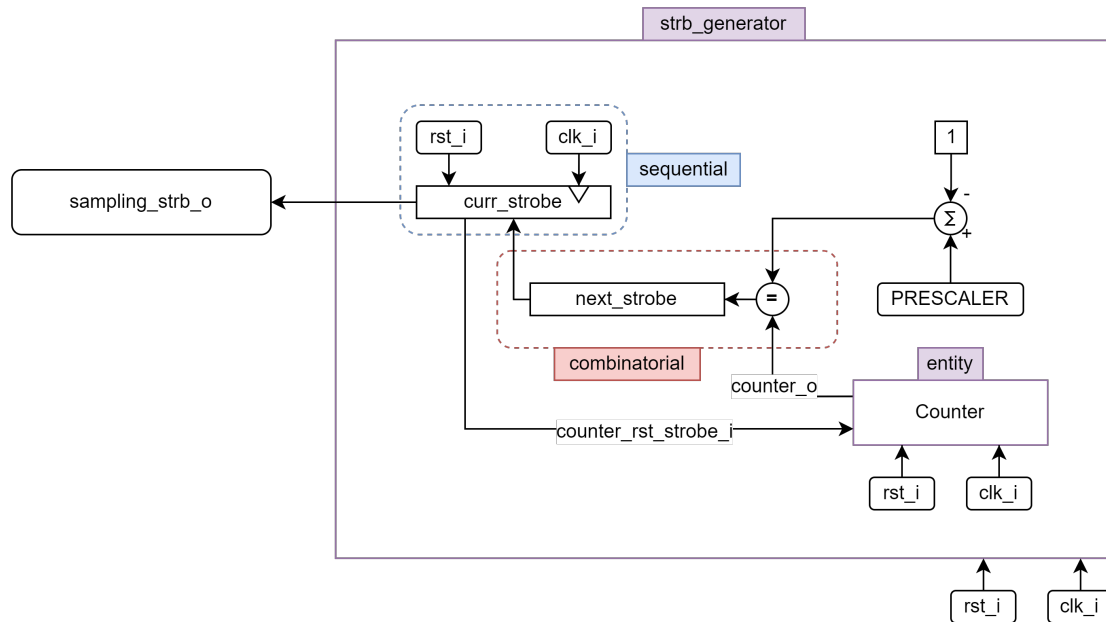
### PWM-Test 1

In dieser PWM-Testbench wird der PWM-Generator statisch, ohne Veränderung des Duty-Cycles betrachtet. Zu sehen ist hier die Konfiguration Period\_counter\_val\_i = 1111 (15) mit dem Arbeitszyklus

ON\_counter\_val\_i = 0110 (6). Da der im Zyklus der Wert 0 mit gezählt wird muss bei einer Periode von 15 also von 0 bis 14 gezählt werden. Nun ist zu sehen, dass der Duty-Cycle von 6 genau sechs Pulse der PWM Periode high ist.



## 4. Strobe Generator



Entity \_\_\_\_\_

Architecture \_\_\_\_\_

Bei dieser Realisierung des Strobe-Generators wurde entschieden, dass die Frequenz, mit der die Strobe Generiert wird keine Zeitdauer, sondern ein Teiler (Prescaler) der Clock ist. Damit kann der sowieso Notwendige Counter auch für die Abzählung der Sampling-Periode verwendet werden.

Signale

```

1 signal next_strobe : std_ulogic := '0';
2 signal curr_strobe : std_ulogic := '0';
3 signal strb_counter : unsigned(PRESCALER-1 downto 0) := (others => '0');

```

(1) reg\_seq : **process** (clk\_i, rst\_i)

Der sequenzielle Prozess bestimmt das Clock- und Resetverhalten. Bei der steigenden Flanke wird der momentanen Strobe curr\_strobe dem nächsten, kombinatorisch ermittelten Strobe-Zustand next\_strobe zugewiesen. Beim Reset wird der Strobe-Ausgang auf '0' gesetzt.

(2) strb\_comb : **process** (strb\_counter)

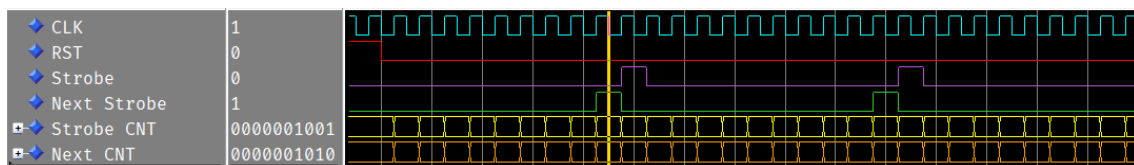
In diesem Prozess wird die Kombinatorik zur Bestimmung des nächsten Strobe-Zustands beschrieben. Standardmäßig ist der nächste Strobe Zustand 'next\_strobe' gleich '0'. Erreicht der Strobe-Counter jedoch seinen maximalen Wert, wird next\_strobe '1'. Da die Reset-Strobe des Counters mit curr\_strobe verbunden ist, wird next\_strobe zurück auf '0' gesetzt. Damit wird erreicht, dass der Strobe-Ausgang genau für eine Periode High ist.

## Simulation

### Strobe Test

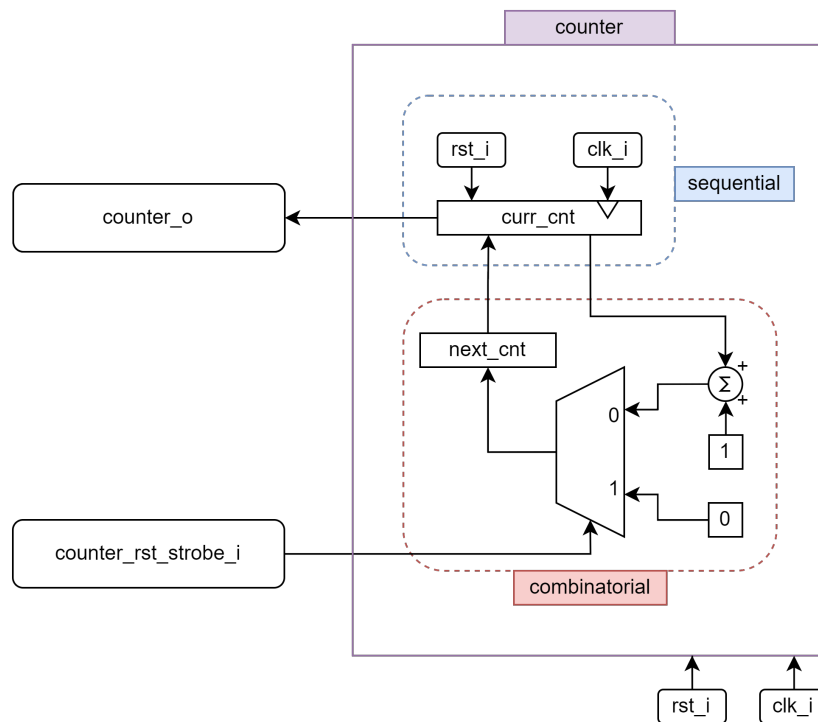
---

Hier ist das Strobe-Signal mit einem Prescaler von 10 zu sehen, wie es nach 10 Taktflanken für eine Periode lang high wird.



Hier ist die Ermittlung der Counter Länge nicht optimal, da sie durch `unsigned(PRESCALER - 1 downto 0)` bestimmt wird. Die Größe des Counter Wertes ist daher viel zu groß, da nur bis zum Prescaler Wert gezählt wird und nicht bis  $2^{10}$ . Man müsste den Prescaler entweder als `std_ulogic_vector` übergeben, oder eine art `log2` Funktion implementieren.

## 5. Counter



Entity \_\_\_\_\_

Architecture \_\_\_\_\_

Signale

```

1 signal curr_cnt : unsigned(COUNTER_LEN-1 downto 0) := (others => '0');
2 signal next_cnt : unsigned(COUNTER_LEN-1 downto 0);

```

(1) `reg_seq : process (clk_i, rst_i)`

Der sequenzielle Prozess bestimmt das Clock- und Reset-Verhalten. Bei der steigenden Flanke wird der momentanen Counter-Wert `curr_cnt` dem nächsten, kombinatorisch ermittelten Counter-Wert `next_cnt` zugewiesen. Beim Reset wird der Counter-Ausgang auf 0 gesetzt.

(2) `cnt_comb : process (curr_cnt, counter_rst_strobe_i)`

Hier erfolgt die kombinatorische Inkrementierung des Counters. `next_cnt` hat den Zustand `curr_cnt + 1`, falls `counter_rst_strobe_i` gerade nicht '1' ist. In diesem Falle wird `next_cnt` synchron auf 0 gesetzt.

Die Simulation für den Counter ist in Vorherigen Übungen bereits mehrmals durchgeführt worden.